

Superscalar processor simulator

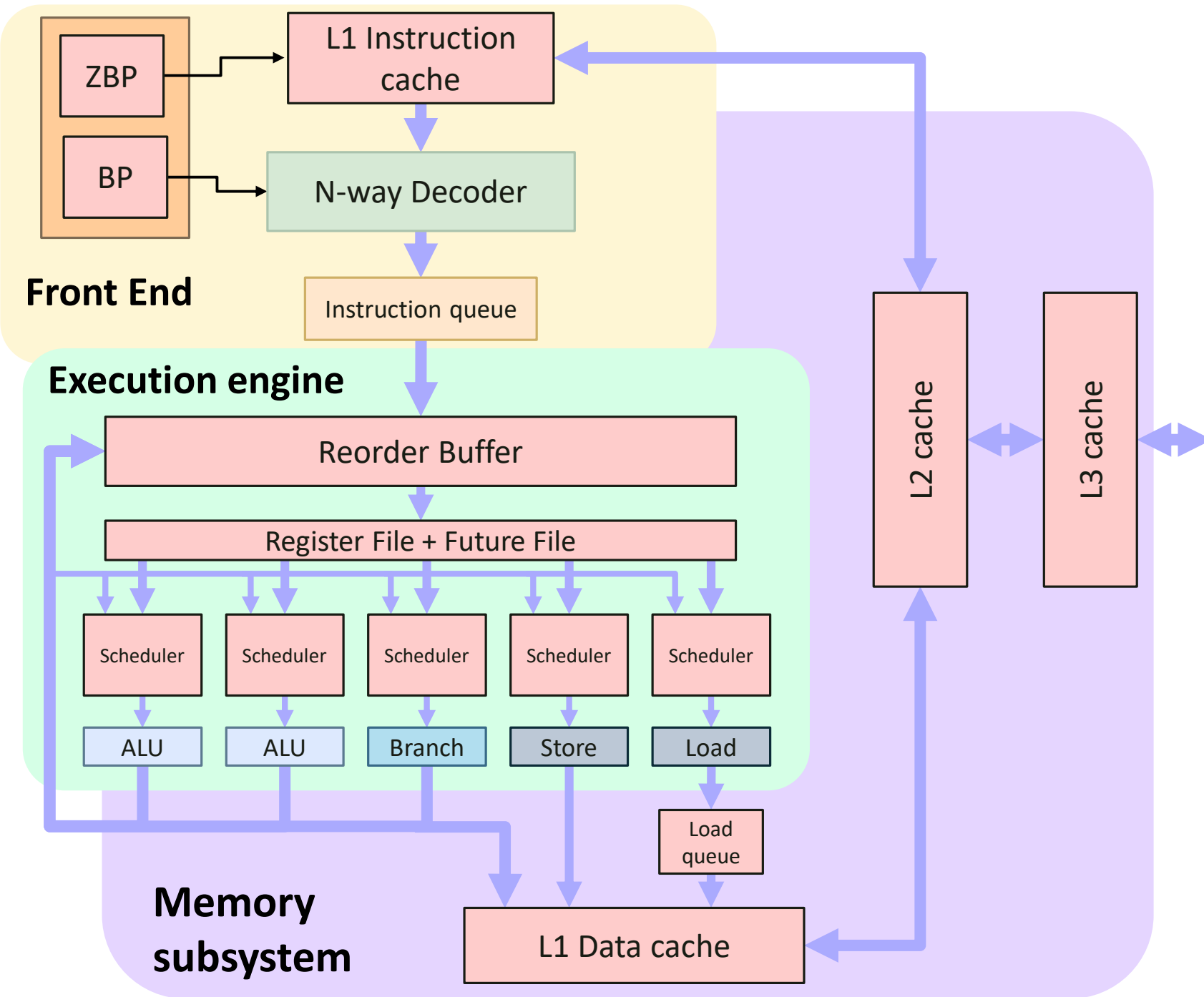
BY MIŁOSZ WĄSACZ



Microarchitecture

A diagram of the simulated processor's microarchitecture

ZBP – Zero-bubble predictor
BP – Branch predictor



Features

- General
 - RISC-V unprivileged ISA
 - Support for all instructions in RV64IM
 - 32 64-bit GPRs + a few CSRs
 - ELF file execution
- 8-stage pipeline
 - Zero-bubble prediction
 - Fetch
 - Branch prediction
 - Decode
 - Issue
 - Execute
 - Write result
 - Commit
- Pipeline features
 - Variable Fetch, Issue & Commit width (default: 4)
 - Variable size decode queue (default: 16)
 - Speculative execution
 - Branch misprediction recovery
 - Precise exceptions
 - Variable ROB & Scheduler capacities (default: 96 & 16 per EU)
- Execution Units
 - Four types: ALU, Branch, Load AGU, Store AGU
 - Multipurpose – a single EU can be of multiple types
- Branch prediction
 - Variable size zero-bubble predictor (default: 1024)
 - Variable capacity 2-bit branch predictor (default: 16K)
- Memory hierarchy
 - Variable size, N-way associative L1I, L1D, L2 & L3 caches (default: 4-way 32KiB, 4-way 32KiB, 8-way 256KiB & 16-way 2MiB)
 - Configurable write policies
- Interactive environment
 - Full overview of the CPU state
 - Step, Run/Continue
 - Capturing *STDOUT* & *STDERR*

PC & Diagnostics

PC: 0x11300

Clock cycle: 450

IPC: 1.07556

Zero-bubble

Accuracy: 83.47%

- ▶ 0x112f0: -
- ▶ 0x112f4: -
- ▶ 0x112f8: -
- ▶ 0x112fc: -

Fetch

- ▶ 0x112e0: 0000000000000011000000010110010011
- ▶ 0x112e4: 000000000000000010100000010000010011
- ▶ 0x112e8: 0000000000000000110100000110000010011
- ▶ 0x112ec: 000000000000000011100000010100010011

Branch predictor

Accuracy: 83.47%

- ▶ 0x112d0: not taken
- ▶ 0x112d4: not taken
- ▶ 0x112d8: not taken
- ▶ 0x112dc: not taken

Instruction Queue

- 0: j -284
- 1:
- 2:
- 3:
- 4:
- 5:
- 6:
- 7:
- 8:
- 9:

Reorder Buffer

- 3: 0x113c4: LOAD s1, 0
- 4: 0x113c8: LOAD s2
- 5: 0x113cc: ALU sp, 1073819480
- 6: 0x113d0: JUMP zero, 0x113d4, 0x101a4
- 7: 0x113d8: LOAD a0
- 8: 0x113dc: ALU a5
- 9: 0x113e0: ALU a3
- 10: 0x113e4: ALU a1
- 11: 0x113e8: ALU a2
- 12:
- 13:
- 14:
- 15:
- 16:
- 17:
- 18:
- 19:
- 20:
- 21:
- 22:
- 23:
- 24:
- 25:
- 26:
- 27:
- 28:

Schedulers

- ALU
- 0: #10, 73728 0, Add
 - 1:
 - 2:
 - 3:
 - 4:
 - 5:
 - 6:
 - 7:
 - 8:
 - 9:
 - 10:
 - 11:

- ALU/MUL
- 0: #11, #8 0, Add
 - 1:
 - 2:
 - 3:
 - 4:
 - 5:
 - 6:
 - 7:
 - 8:
 - 9:
 - 10:
 - 11:

- BRANCH
- 0:
 - 1:
 - 2:
 - 3:
 - 4:
 - 5:
 - 6:
 - 7:
 - 8:
 - 9:
 - 10:
 - 11:

Load queue

- 0: #7, 0x12990

Register File

zero:	ra:	sp:	gp:	tp:	t0:	t1:	t2:	s0:	s1:	a0:	a1:	a2:	a3:	a4:	a5:	a6:	a7:	s2:
0	65956	107381	75800	0	70708	15	0	0	2	0	72284	0	76864	73728	0	31	0	2

Future File

zero:	ra:	sp:	gp:	tp:	t0:	t1:	t2:	s0:	s1:	a0:	a1:	a2:	a3:	a4:	a5:	a6:	a7:	s2:
0	65956	107381	75800	0	70708	15	0	0	0	0	72284	0	76864	73728	0	31	0	2
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	#7	#10	#11	#9	✓	#8	✓	✓	#4

Project goals

- Achieved

- Accurate simulation
 - Research into real latencies of specific components
- Ease of use
 - Interactive environment
 - Emulation of basic Linux system calls for printing and memory allocation
- Working with real low-level specification (RISC-V ISA), not a made-up toy ISA
- Eliminating the “imperative side” of the code
 - Mostly achieved, uses a model of buffered registers that only get updated at the end of a simulated clock cycle

- In Progress/Planned

- Privileged ISA support
 - Start with a simple custom kernel
 - Eventually add support for the Linux kernel
- Improving the external APIs
 - Further elimination of the imperative aspects of the simulation, by encapsulation within individual components
- Extensive unit and integration testing
- Better configuration options
 - Defining the model using config files
 - Support for external components using dynamic linking

Questions
